

```
In [42]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import duckdb

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE #Help with class imbalance for a classifie

from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression

import pickle
```

```
In [22]: sql_departments = duckdb.sql("SELECT * FROM 'departments.csv'")
sql_encounters = duckdb.sql("SELECT * FROM 'encounters.csv'")
sql_patients = duckdb.sql("SELECT * FROM 'patients.csv'")
sql_diagnosis = duckdb.sql("SELECT * FROM 'diagnosis.csv'")
sql_providers = duckdb.sql("SELECT * FROM 'providers.csv'")
sql_social_determinant = duckdb.sql("SELECT * FROM 'social_determinants.csv'")
sql_tiger = duckdb.sql("SELECT * FROM 'tigercensuscodes.csv'")
```

```
In [23]: combined_data = duckdb.sql("""
    WITH duped_diagnosis AS (
        SELECT *
        FROM (
            SELECT *,
                ROW_NUMBER() OVER (PARTITION BY DiagnosisKey ORDER BY DiagnosisKey)
            FROM sql_diagnosis
        )
        WHERE rn = 1
    ),
    duped_providers AS (
        SELECT *
        FROM (
            SELECT *,
                ROW_NUMBER() OVER (PARTITION BY DurableKey, Type ORDER BY DurableKey)
            FROM sql_providers
        )
        WHERE rn = 1
    ),
    duped_patients AS (
        SELECT *
        FROM (
            SELECT *,
                ROW_NUMBER() OVER (PARTITION BY DurableKey ORDER BY DurableKey) AS
```

```

        FROM sql_patients
    )
    WHERE rn = 1
),
duped_departments AS (
    SELECT *
    FROM (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY DepartmentKey ORDER BY DepartmentKey)
        FROM sql_departments
    )
    WHERE rn = 1
)
SELECT
    e.*,
    p.CensusBlockGroupFipsCode, p.FirstRace, p.MaritalStatus, p.MyChartStatus,
    p.OmbEthnicity, p.OmbRace, p.SexAssignedAtBirth, p.SexualOrientation,
    p.SmokingStatus, p.VitalStatus, p.PatientBirthYearBin,
    d.GroupName, d.GroupCode, d.DiagnosisName, d.DiagnosisValue,
    pr.ClinicianTitle, pr.OfficeAddress, pr.OfficeCity, pr.OfficePostalCode,
    pr.PrimaryDepartment, pr.PrimarySpecialty,
    de.Address, de.City, de.County, de.DepartmentName,
    de.DepartmentSpecialty, de.DepartmentType, de.PostalCode, de.CensusTract
FROM sql_encounters AS e
LEFT JOIN duped_patients AS p
    ON e.PatientDurableKey = p.DurableKey
LEFT JOIN duped_diagnosis AS d
    ON e.PrimaryDiagnosisKey = d.DiagnosisKey
LEFT JOIN duped_providers AS pr
    ON e.ProviderDurableKey = pr.DurableKey AND e.Type = pr.Type
LEFT JOIN duped_departments AS de
    ON e.DepartmentKey = de.DepartmentKey
""")

duckdb.sql("SELECT COUNT(*) FROM combined_data")

```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

Out[23]:

count_star() int64
7675801

Looking at Emergency Admissions

```

In [24]: pd.set_option('display.max_colwidth', None)
duckdb.sql("""
    SELECT GroupName, COUNT(*) as Count
    FROM combined_data
    WHERE IsEdVisit = '1'
    GROUP BY GroupName
    ORDER BY Count DESC
""").df().head(10)

```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

Out[24]:

	GroupName	Count
0	Pain in throat and chest	12023
1	Abdominal and pelvic pain	11956
2	Persons encountering health services for specific procedures and treatment, not carried out	7376
3	Dorsalgia	7070
4	Other sepsis	6838
5	Other joint disorder, not elsewhere classified	6230
6	Nausea and vomiting	4551
7	Other and unspecified soft tissue disorders, not elsewhere classified	4259
8	COVID-19	3711
9	Open wound of head	3599

```
In [73]: emergency_df = duckdb.sql("""
        SELECT *
        FROM combined_data
        WHERE IsEdVisit = '1'
        """).df()
        emergency_df.shape[0]
```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

Out[73]: 261458

```
In [27]: emergency_df = emergency_df[emergency_df["GroupName"].isin(["Abdominal and pelvic p
        emergency_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 23979 entries, 12 to 261383
```

```
Data columns (total 60 columns):
```

#	Column	Non-Null Count	Dtype
0	Date	23979 non-null	datetime64[us]
1	AdmissionInstant	23979 non-null	object
2	AdmitYear	23979 non-null	object
3	AdmitMonth	23979 non-null	object
4	AdmitDay	23979 non-null	object
5	AdmitHour	23979 non-null	object
6	AdmitMinute	23979 non-null	object
7	AdmissionSource	23976 non-null	object
8	AdmissionType	23979 non-null	object
9	DischargeInstant	23979 non-null	object
10	DischargeYear	23979 non-null	object
11	DischargeMonth	23979 non-null	object
12	DischargeDay	23979 non-null	object
13	DischargeHour	23979 non-null	object
14	DischargeMinute	23979 non-null	object
15	EncounterKey	23979 non-null	int64
16	PatientDurableKey	23979 non-null	int64
17	Type	23979 non-null	object
18	VisitType	23979 non-null	object
19	VisitTypeDescription	23979 non-null	object
20	ProviderDurableKey	23979 non-null	int64
21	AttendingProviderDurableKey	23979 non-null	int64
22	DischargeProviderDurableKey	23979 non-null	int64
23	DepartmentKey	23979 non-null	int64
24	PrimaryDiagnosisKey	23979 non-null	int64
25	IsEdVisit	23979 non-null	int64
26	IsHospitalAdmission	23979 non-null	int64
27	IsHospitalOutpatientVisit	23979 non-null	int64
28	IsInpatientAdmission	23979 non-null	int64
29	IsObservation	23979 non-null	int64
30	IsOutpatientFaceToFaceVisit	23979 non-null	int64
31	CensusBlockGroupFipsCode	23445 non-null	object
32	FirstRace	23445 non-null	object
33	MaritalStatus	23445 non-null	object
34	MyChartStatus	23445 non-null	object
35	OmbEthnicity	23445 non-null	object
36	OmbRace	23445 non-null	object
37	SexAssignedAtBirth	23445 non-null	object
38	SexualOrientation	23445 non-null	object
39	SmokingStatus	23445 non-null	object
40	VitalStatus	23445 non-null	object
41	PatientBirthYearBin	23445 non-null	object
42	GroupName	23979 non-null	object
43	GroupCode	23979 non-null	object
44	DiagnosisName	23979 non-null	object
45	DiagnosisValue	23979 non-null	object
46	ClinicianTitle	0 non-null	object
47	OfficeAddress	0 non-null	object
48	OfficeCity	0 non-null	object
49	OfficePostalCode	0 non-null	object
50	PrimaryDepartment	0 non-null	object

```

51 PrimarySpecialty          0 non-null    object
52 Address                   23979 non-null object
53 City                      23979 non-null object
54 County                   23979 non-null object
55 DepartmentName           23979 non-null object
56 DepartmentSpecialty       23979 non-null object
57 DepartmentType           23979 non-null object
58 PostalCode                23979 non-null object
59 CensusTract               23979 non-null object

```

```
dtypes: datetime64[us](1), int64(13), object(46)
```

```
memory usage: 11.2+ MB
```

```

In [76]: drop_cols = [
# All null
'ClinicianTitle', 'OfficeAddress', 'OfficeCity', 'OfficePostalCode',
'PrimaryDepartment', 'PrimarySpecialty',
# Redundant time
'AdmissionInstant', 'DischargeInstant',
'AdmitYear', 'AdmitMonth', 'AdmitDay', 'AdmitHour', 'AdmitMinute',
'DischargeYear', 'DischargeMonth', 'DischargeDay', 'DischargeHour', 'DischargeM
# Redundant keys
'AttendingProviderDurableKey', 'DischargeProviderDurableKey'
]

emergency_df = emergency_df.drop(columns=drop_cols)
print(emergency_df.shape)

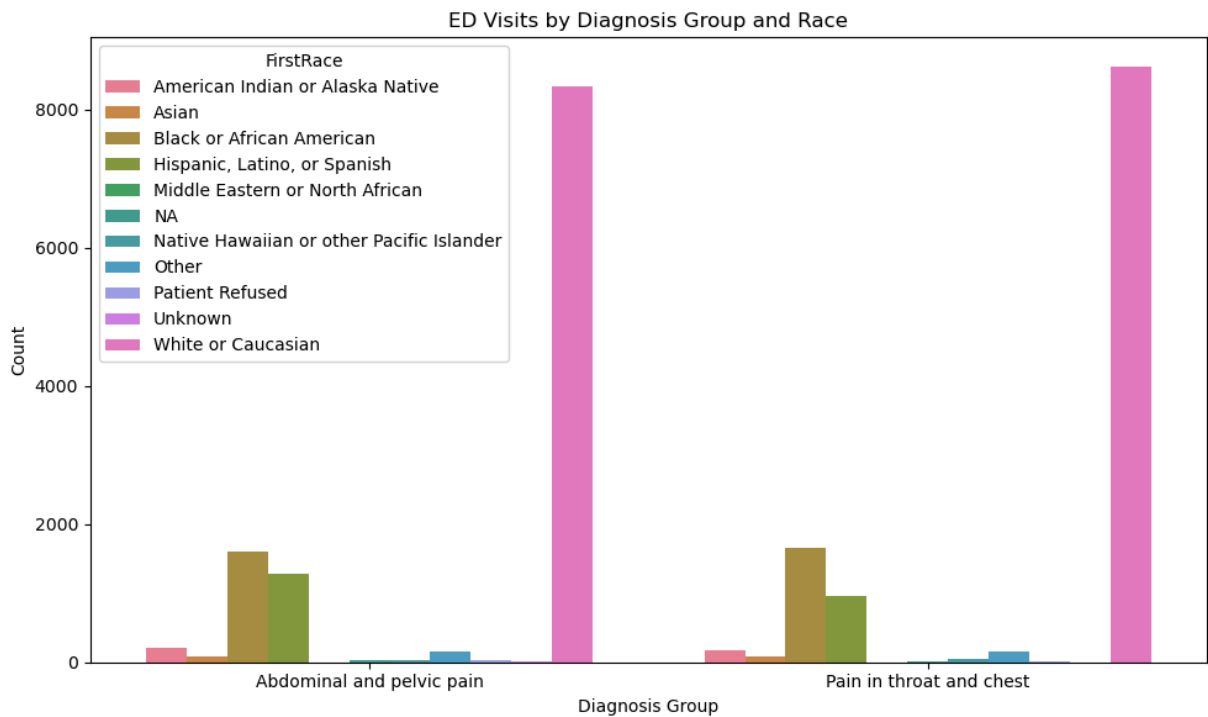
```

```
(261458, 40)
```

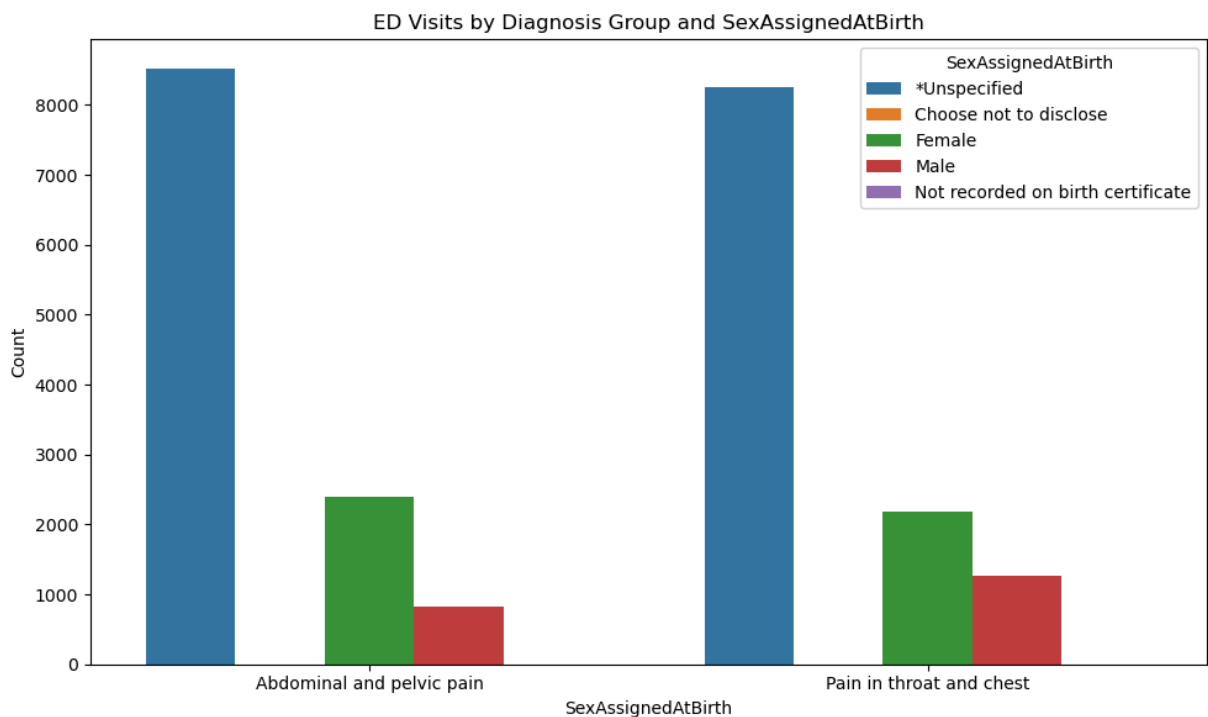
```

In [29]: grouped_ed = emergency_df.groupby(["GroupName", "FirstRace"]).size().reset_index(name="Count")
plt.figure(figsize=(10, 6))
sns.barplot(data=grouped_ed, x="GroupName", y="Count", hue="FirstRace")
plt.title("ED Visits by Diagnosis Group and Race")
plt.xlabel("Diagnosis Group")
plt.ylabel("Count")
plt.xticks(wrap=True)
plt.legend(title="FirstRace")
plt.tight_layout()
plt.show()

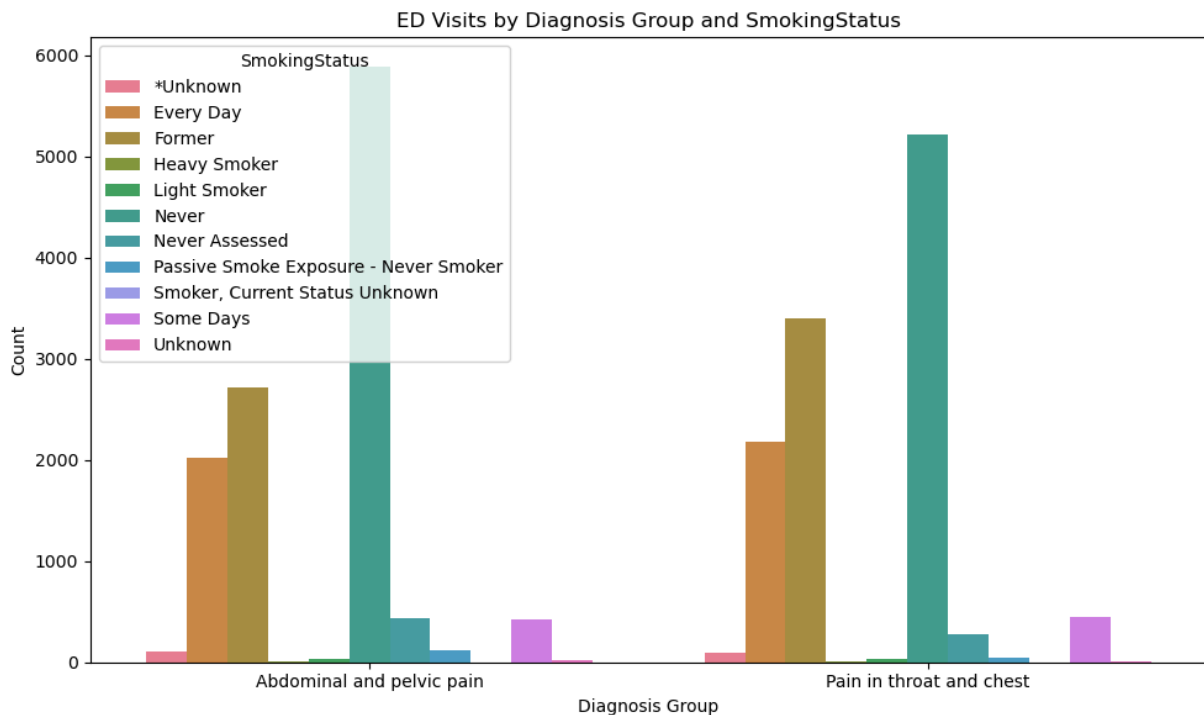
```



```
In [31]: grouped_ed = emergency_df.groupby(["GroupName", "SexAssignedAtBirth"]).size().reset
plt.figure(figsize=(10, 6))
sns.barplot(data=grouped_ed, x="GroupName", y="Count", hue="SexAssignedAtBirth")
plt.title("ED Visits by Diagnosis Group and SexAssignedAtBirth")
plt.xlabel("SexAssignedAtBirth")
plt.ylabel("Count")
plt.xticks(wrap=True)
plt.legend(title="SexAssignedAtBirth")
plt.tight_layout()
plt.show()
```



```
In [32]: grouped_ed = emergency_df.groupby(["GroupName", "SmokingStatus"]).size().reset_index()
plt.figure(figsize=(10, 6))
sns.barplot(data=grouped_ed, x="GroupName", y="Count", hue="SmokingStatus")
plt.title("ED Visits by Diagnosis Group and SmokingStatus")
plt.xlabel("Diagnosis Group")
plt.ylabel("Count")
plt.xticks(wrap=True)
plt.legend(title="SmokingStatus")
plt.tight_layout()
plt.show()
```



```
In [34]: emergency_df["Month"] = emergency_df["Date"].dt.month
emergency_df["Year"] = emergency_df["Date"].dt.year
emergency_df["YearMonth"] = emergency_df["Date"].dt.to_period("M")

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()

years = [2022, 2023, 2024, 2025]

for i, year in enumerate(years):
    year_df = emergency_df[emergency_df["Year"] == year]
    grouped = year_df.groupby(["Month", "GroupName"]).size().reset_index(name="Count")
    grouped = grouped.sort_values("Month")

    for group, data in grouped.groupby("GroupName"):
        axes[i].plot(data["Month"], data["Count"], marker='o', label=group)

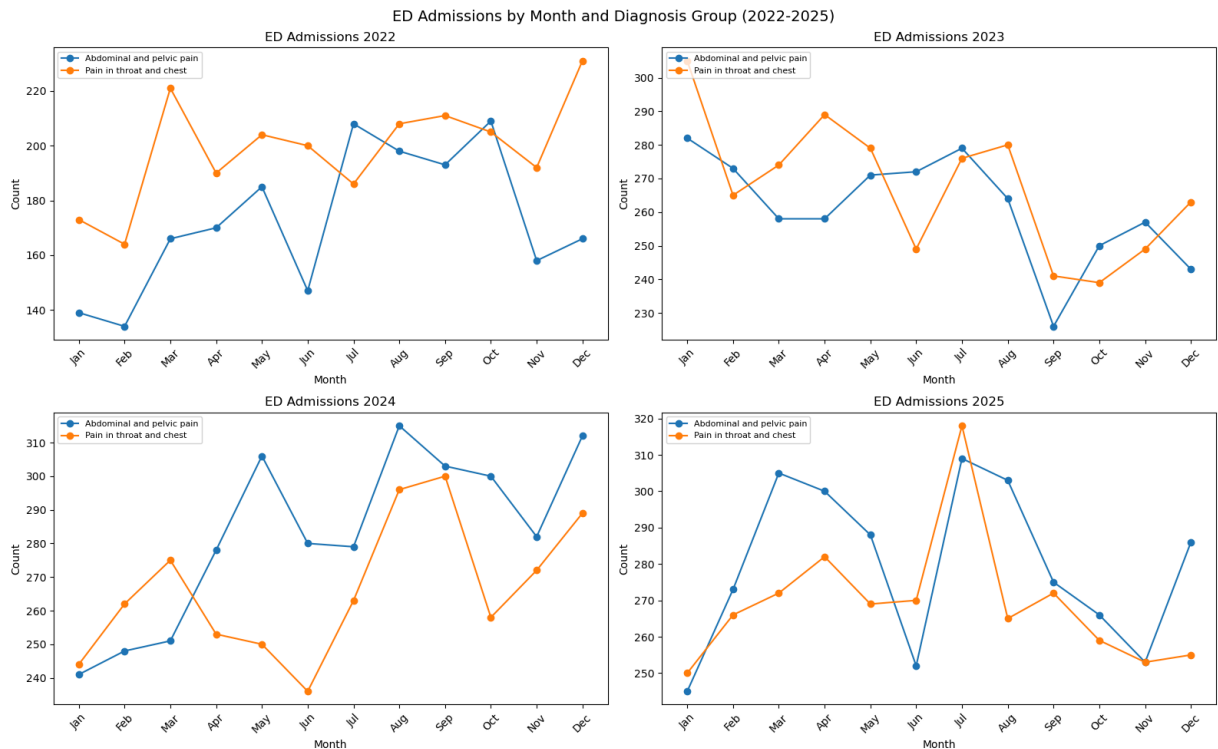
    axes[i].set_title(f"ED Admissions {year}")
    axes[i].set_xlabel("Month")
    axes[i].set_ylabel("Count")
    axes[i].set_xticks(range(1, 13))
    axes[i].set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep',
```

```

axes[i].legend(loc='upper left', fontsize=8)

plt.suptitle("ED Admissions by Month and Diagnosis Group (2022-2025)", fontsize=14)
plt.tight_layout()
plt.show()

```



From our visualization there from times around June and around fall (October November) the admission counts for these two issues occur the most around these seasons. I think running a *PCA analysis* and understanding what variables highly contribute to these cases can help us further understand this trend we are observing within the data.

```

In [36]: pca_df = emergency_df.copy()

cat_cols = [
    'GroupName', 'AdmissionType', 'AdmissionSource', 'Type',
    'VisitType', 'FirstRace', 'MaritalStatus', 'OmbEthnicity',
    'OmbRace', 'SexAssignedAtBirth', 'SmokingStatus',
    'VitalStatus', 'PatientBirthYearBin', 'DepartmentSpecialty'
]

feature_cols = [
    'GroupName', 'AdmissionType', 'AdmissionSource', 'Type',
    'VisitType', 'FirstRace', 'MaritalStatus', 'OmbEthnicity',
    'OmbRace', 'SexAssignedAtBirth', 'SmokingStatus',
    'VitalStatus', 'PatientBirthYearBin', 'DepartmentSpecialty',
    'IsEdVisit', 'IsHospitalAdmission', 'IsInpatientAdmission',
    'IsObservation', 'IsOutpatientFaceToFaceVisit', 'Month', 'Year'
]

label = LabelEncoder()
for col in cat_cols:
    if col in pca_df.columns:

```

```

pca_df[col] = label.fit_transform(pca_df[col].astype(str))

X = pca_df[feature_cols].fillna(0)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA()
pca.fit(X_scaled)

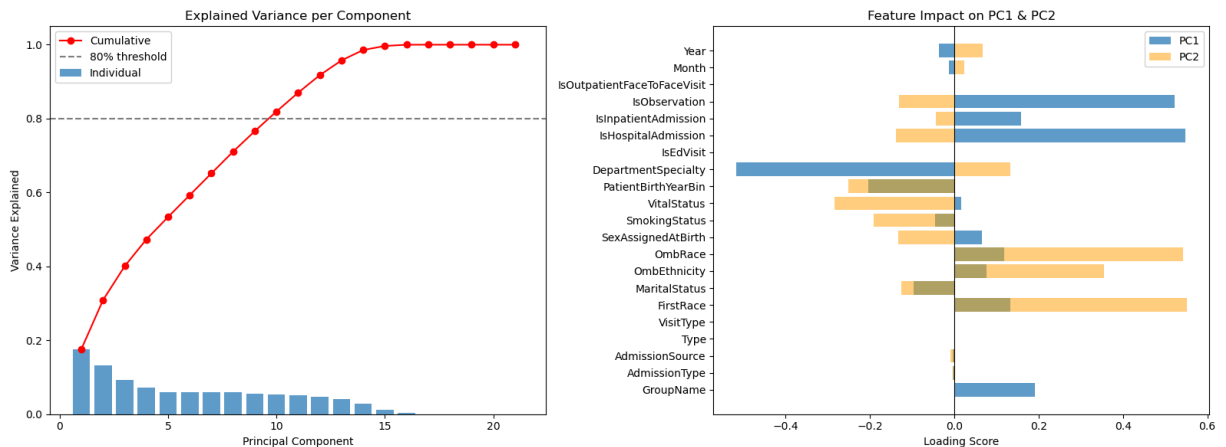
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
explained = pca.explained_variance_ratio_
cumulative = np.cumsum(explained)

axes[0].bar(range(1, len(explained)+1), explained, alpha=0.7, label='Individual')
axes[0].plot(range(1, len(explained)+1), cumulative, marker='o', color='red', label='Cumulative')
axes[0].axhline(y=0.8, color='gray', linestyle='--', label='80% threshold')
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Variance Explained')
axes[0].set_title('Explained Variance per Component')
axes[0].legend()

loadings = pca.components_[1:2].T
axes[1].barh(feature_cols, loadings[:, 0], alpha=0.7, label='PC1')
axes[1].barh(feature_cols, loadings[:, 1], alpha=0.5, label='PC2', color='orange')
axes[1].set_xlabel('Loading Score')
axes[1].set_title('Feature Impact on PC1 & PC2')
axes[1].axvline(x=0, color='black', linewidth=0.8)
axes[1].legend()

plt.tight_layout()
plt.show()

```



Lets see the admission rate for people after going to the ED

```

In [80]: monthly_admission = duckdb.sql("""
SELECT
    DATE_TRUNC('month', Date) AS Month,
    COUNT(*) AS Total_Visits,
    SUM(IsHospitalAdmission) AS Total_Admissions
FROM combined_data
WHERE IsEdVisit = 1
GROUP BY DATE_TRUNC('month', Date)

```

```
ORDER BY Month
""").df()
```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

```
In [81]: from statsmodels.tsa.holtwinters import ExponentialSmoothing

fig, axes = plt.subplots(2, 1, figsize=(14, 8))

axes[0].plot(monthly_admissions['YearMonth'], monthly_admissions['Total_Visits'], m
axes[0].set_title('Monthly ED Visits (2022-2025)')
axes[0].set_ylabel('Total Visits')
axes[0].tick_params(axis='x', rotation=45)
axes[0].grid(True, alpha=0.3)

axes[1].plot(monthly_admissions['YearMonth'], monthly_admissions['Admission_Rate'],
axes[1].set_title('Monthly Admission Rate % (2022-2025)')
axes[1].set_ylabel('Admission Rate %')
axes[1].tick_params(axis='x', rotation=45)
axes[1].grid(True, alpha=0.3)

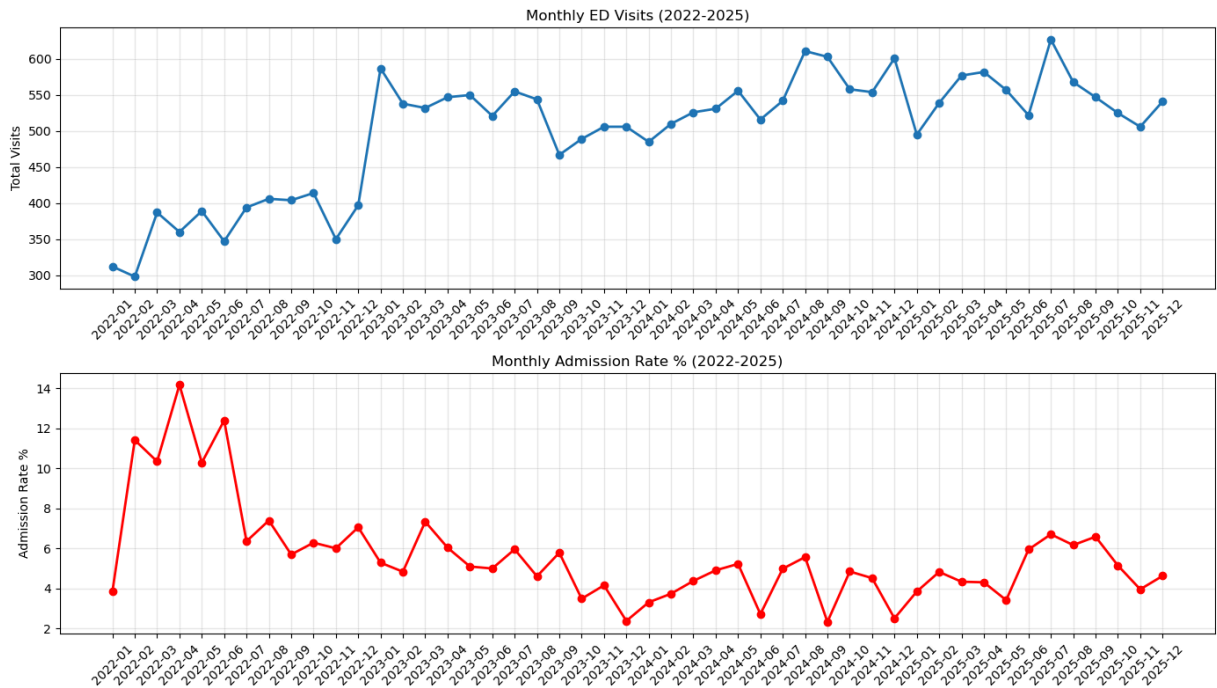
plt.tight_layout()
plt.show()

#Forecast the next 6 months
model_visits = ExponentialSmoothing(
    monthly_admissions['Total_Visits'],
    trend='add',
    seasonal='add',
    seasonal_periods=12
).fit()

model_admissions = ExponentialSmoothing(
    monthly_admissions['Admission_Rate'],
    trend='add',
    seasonal='add',
    seasonal_periods=12
).fit()

forecast_visits = model_visits.forecast(6)
forecast_admissions = model_admissions.forecast(6)

print("=== 6 Month Forecast ===")
for i, (v, a) in enumerate(zip(forecast_visits, forecast_admissions)):
    print(f"Month {i+1}: {v:.0f} visits | {a:.2f}% admission rate")
```



=== 6 Month Forecast ===

Month +1:	563 visits		3.59% admission rate
Month +2:	564 visits		6.02% admission rate
Month +3:	598 visits		6.70% admission rate
Month +4:	598 visits		7.73% admission rate
Month +5:	606 visits		6.65% admission rate
Month +6:	569 visits		7.41% admission rate

This is very interesting to see but it makes a lot of sense. During 2022 that was covid era meaning the admission rates to hospitalize patients that had Covid-19 were in the rise at the time. However, though the rate receded overtime, the total amount of people admitted to the hospitals remained the same and did not return to levels before covid

TRAINING RandomForrest & Logistic Regression

Exploring Hosipital Admissions

DESCRIPTION:

```
In [90]: emergency_df = duckdb.sql("""
SELECT *
FROM combined_data
WHERE IsEdVisit = '1'
""").df()
```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

```
In [91]: print(f"Admitted: {(emergency_df['IsHospitalAdmission'] == 1).sum()}")
print(f"Not Admitted:{(emergency_df['IsHospitalAdmission'] == 0).sum()}")
```

Admitted: 54135
Not Admitted:207323

This isn't good. We have a massive class imbalance where more than 90% of the encounters are either not admitted to the hospital. We will be using *SMOTE* package to resample and balance the classifiers

```
In [92]: drop_cols = [
    # All null
    'ClinicianTitle', 'OfficeAddress', 'OfficeCity', 'OfficePostalCode',
    'PrimaryDepartment', 'PrimarySpecialty',
    # Redundant time
    'AdmissionInstant', 'DischargeInstant',
    'AdmitYear', 'AdmitMonth', 'AdmitDay', 'AdmitHour', 'AdmitMinute',
    'DischargeYear', 'DischargeMonth', 'DischargeDay', 'DischargeHour', 'DischargeM
    # Redundant keys
    'AttendingProviderDurableKey', 'DischargeProviderDurableKey'
]

emergency_df = emergency_df.drop(columns=drop_cols)

emergency_df["Month"] = emergency_df["Date"].dt.month
emergency_df["Year"] = emergency_df["Date"].dt.year
emergency_df["YearMonth"] = emergency_df["Date"].dt.to_period("M")
```

```
In [93]: full_pca_df = emergency_df.copy()

cat_cols = [
    'GroupName', 'OmbRace', 'FirstRace', 'OmbEthnicity',
    'SexAssignedAtBirth', 'SmokingStatus', 'MaritalStatus',
    'PatientBirthYearBin'
]

le = LabelEncoder()
for col in cat_cols:
    if col in full_pca_df.columns:
        full_pca_df[col] = le.fit_transform(full_pca_df[col].astype(str))

feature_cols = [
    'GroupName', 'OmbRace', 'FirstRace', 'OmbEthnicity',
    'SexAssignedAtBirth', 'SmokingStatus', 'MaritalStatus',
    'PatientBirthYearBin', 'Month', 'Year'
]

X = full_pca_df[feature_cols].fillna(0)
y = full_pca_df['IsHospitalAdmission']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=67, stratify=y
)

smote = SMOTE(random_state=67)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print("After SMOTE:")
print(f"Class 0: {sum(y_train_sm == 0)}")
print(f"Class 1: {sum(y_train_sm == 1)}")
```

After SMOTE:

Class 0: 165858

Class 1: 165858

```
In [94]: lr_model = LogisticRegression(max_iter = 5000, class_weight="balanced", random_state=42)
lr_cv = cross_val_score(lr_model, X_train_sm, y_train_sm, cv=5, scoring='accuracy')
```

```
forest_model = RandomForestClassifier(n_estimators=100, class_weight="balanced", random_state=42)
forest_cv = cross_val_score(forest_model, X_train_sm, y_train_sm, scoring="accuracy", cv=5)
```

```
In [96]: print("--Logistic Regression Classifier--")
print(f"CV Scores: {lr_cv.round(4)}")
print(f"CV Average: {lr_cv.mean().round(4)}")
print(f"CV Standard dev: {lr_cv.std().round(4)}")

print("--RandomForest Classifier--")
print(f"CV Scores: {forest_cv.round(4)}")
print(f"CV Average: {forest_cv.mean().round(4)}")
print(f"CV standard dev: {forest_cv.std().round(4)}")
```

```
--Logistic Regression Classifier--
CV Scores: [0.6799 0.6846 0.6878 0.689 0.6846]
CV Average: 0.6852
CV Standard dev: 0.0032
--RandomForest Classifier--
CV Scores: [0.8156 0.8731 0.8986 0.8972 0.8968]
CV Average: 0.8762
CV standard dev: 0.0318
```

```
In [97]: forest_model.fit(X_train_sm, y_train_sm)
lr_model.fit(X_train_sm, y_train_sm)

y_pred_rf = forest_model.predict(X_test)
y_pred_lr = lr_model.predict(X_test)

print("-- Random Forest--")
print(classification_report(y_test, y_pred_rf, target_names=['Not Admitted', 'Admitted']))

print("-- Logistic Regression --")
print(classification_report(y_test, y_pred_lr, target_names=['Not Admitted', 'Admitted']))

# --- Feature Importance ---
importance_df = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': forest_model.feature_importances_
}).sort_values('Importance', ascending=False)

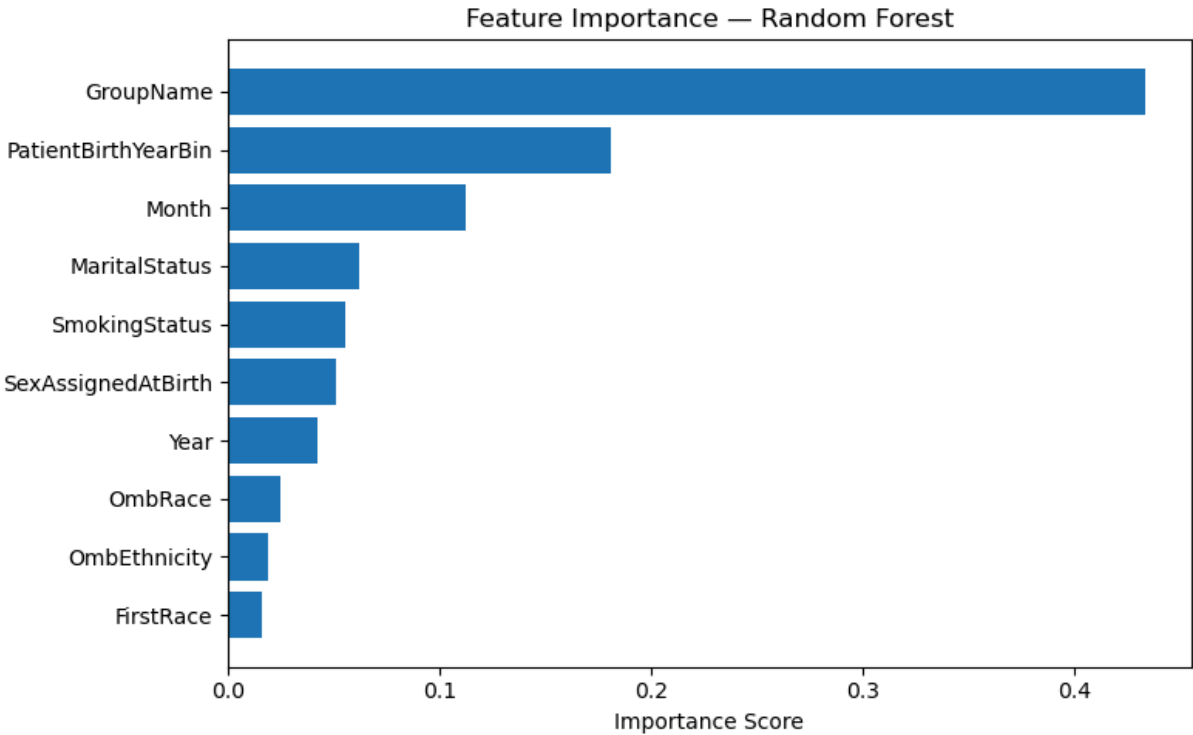
plt.figure(figsize=(8, 5))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.title('Feature Importance - Random Forest')
plt.xlabel('Importance Score')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

=== Random Forest ===

	precision	recall	f1-score	support
Not Admitted	0.90	0.87	0.89	41465
Admitted	0.57	0.64	0.60	10827
accuracy			0.82	52292
macro avg	0.73	0.75	0.74	52292
weighted avg	0.83	0.82	0.83	52292

=== Logistic Regression ===

	precision	recall	f1-score	support
Not Admitted	0.90	0.63	0.74	41465
Admitted	0.34	0.72	0.46	10827
accuracy			0.65	52292
macro avg	0.62	0.67	0.60	52292
weighted avg	0.78	0.65	0.68	52292



In []: